项目经历

YOLOv8 + OpenVINO 端到端部署（CPU 推理与 INT8 量化） 2026.10

个人项目　代码：github.com/liumaker/yolov8-openvino-deploy

- **导出与转换**：打通 PyTorch → ONNX（opset 12，静态/动态轴）→ OpenVINO IR（fp32/fp16）链路，用 Netron 核对计算图；以同一输入张量对齐三端原始输出，相对误差 3.2×10^{-6}，证明导出无语义损失。
- **自研前后处理**：不依赖 Ultralytics 高层 API，手写 letterbox、输出解码（$84 = 4 + 80$，anchor-free）与按类 NMS，与官方实现逐框对齐（5/5 框匹配，坐标偏差 0.0001 px），单帧端到端 56.9 ms。
- **INT8 量化**：用 NNCF 做 PTQ（300 张校准图）。朴素量化掉点严重（−5.9 AP），改用 accuracy-aware 逐层回退，COCO val2017 500 张上 mAP 0.3998 → 0.3886（−1.1 点，−2.8%），单帧延迟 30.7 → 16.6 ms（−46%），吞吐 32 → 57 FPS。
- **并发与工程化**：benchmark_app 对比 FP32/FP16/INT8 延迟与吞吐；实现多路视频流并发（每路独立 `InferRequest`），4 路 INT8 总吞吐 48 FPS（约为 FP32 的 1.9×），进程 CPU ∼ 630%，定位瓶颈为 CPU 算力；沉淀 10 个可复现脚本与自动化验收（数值对齐 / mAP / 基准）。

技术栈：Python ・PyTorch ・ONNX / ONNX Runtime ・OpenVINO ・NNCF ・OpenCV ・pycocotools